~/presentation/start.sh

HOCHSCHULE FULDA
UNIVERSITY OF APPLIED SCIENCES

Parallel Programming

AI5085 (WiSe25/26)

GROUP 11

EF **Ehtesham Faraz**
Presenter

MF **Md Fuzail**
Presenter

Slide 1: Sliding Window Sums & Hotspots

Speaker : Md Fuzail

Strategy Overview

Problem
Parallel analysis of 2D heatmap — Sliding Window Sums & Hotspots.

Approach
Data parallelism via Row-Major & Column-Major Traversal.

Memory Efficiency
Chunking arrays spatially to maximize cache locality and minimize misses.

Scheduling
`schedule(static)` mapped for uniformly distributed workloads.

Hashing Loop
Collapsed 2D boundaries into a 1D flat loop for compiler optimization.

Thread Safety
`reduction(+:var)` prevents race conditions on shared counters.

</> heatmap_analysis.c

```
// Initialize matrix A with random values — Ehtesham
#pragma omp parallel for schedule(static)
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < columns; ++j) {
        /* generate & assign my_rand() */
    }
}

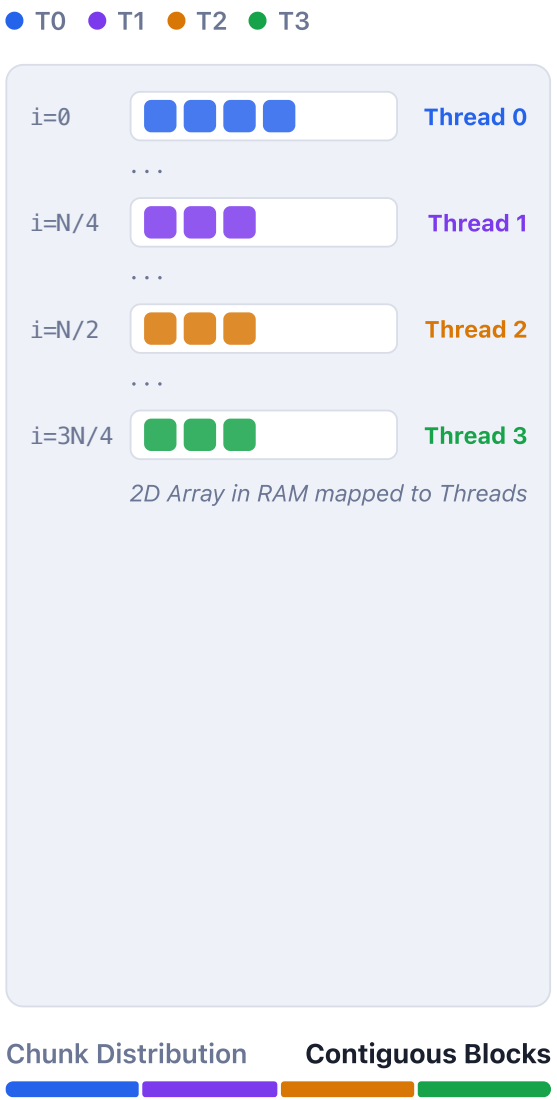
// Hash A → B applying work_factor iterations — Md Fu
#pragma omp parallel for schedule(static)
for (size_t k = 0; k < total_cells; ++k) {
    /* compute hash(val) w times */
    B[k] = val;
}

// Max sliding window sums per column — Ehtesham Fara
#pragma omp parallel for schedule(static)
for (int j = 0; j < columns; ++j) {
    /* slide window down column j to find max */
    max_sums[j] = max_sum;
}

// Count hotspots with reduction — Md Fuzail
#pragma omp parallel for reduction(+:total_hotspots)
for (int i = 0; i < rows; ++i) {
    /* check 4 neighbours: up, down, left, right */
    hotspots_per_row[i] = row_count;
}
```

Memory Access

Static Schedule



Slide 1: Hotspot Detection & Early Exit Strategy

Speaker : Md Fuzail

Objectives & Challenges

Goal

Terminate immediately upon finding a row with **no local hotspots**.

Challenge

Standard C `break` is illegal in OpenMP loops. We must break a parallel loop barrier instantly without leaving zombie threads.

Solution: OpenMP Cancellation

`#pragma omp cancel for` `#pragma omp cancellation point for`

Synchronization

Used `#pragma omp critical` to safely set `early_exit` and `bad_row`, ensuring only the first offending row is reported.

Runtime Requirement

OpenMP disables cancellation by default for performance. We must set `export OMP_CANCELLATION=true` before execution.



heatmap_analysis_quick.c

```c
int early_exit = 0;  bad_row = -1;

#pragma omp parallel
{
    #pragma omp for schedule(static)
    for (int i = 0; i < rows; ++i) {
        #pragma omp cancellation point for

        int row_hotspots = 0;
        /* ... count local hotspots in row i ... */

        if (row_hotspots == 0) {
            #pragma omp critical
            {
                if (!early_exit) {
                    early_exit = 1;  bad_row = i;
                    printf("Row %d contains no hotspots.\n", i);
                    printf("Early exit.\n");
                }
            }
            #pragma omp cancel for
        }
    }
}
```

Slide 1:– OpenMP Task Parallelism

Speaker: Ehtesham Faraz

Strategy Overview

Problem

Compute π via numerical integration using a dynamic, load-imbalanced recursive task tree.

Approach

OpenMP task parallelism — one thread seeds the recursion via `#pragma omp single`; all threads steal tasks from the pool.

Recursion

Each task computes π over random steps, then spawns **1–4 child tasks** until `g_num_tasks` is reached.

Atomics

Thread-safe accumulation of `pi_sum` and per-thread counters; unique task IDs via `atomic capture`.

Key Pragmas

`#pragma omp task`

`#pragma omp single`

`#pragma omp atomic`

pi_tasks.c

```
/* Entry point — one thread seeds the whole tree */
#pragma omp parallel
{
    #pragma omp single
    {
        total_tasks_created = 1;
        #pragma omp task firstprivate(seed)
        pi_task(seed);
    }
}

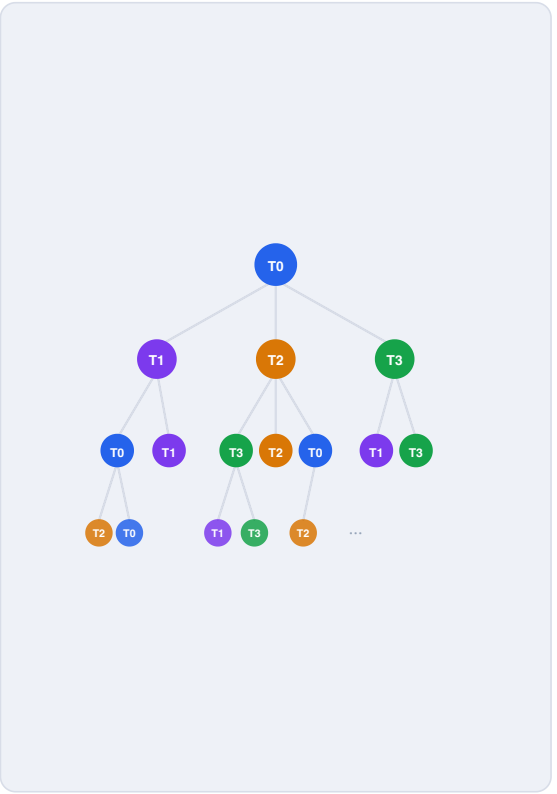
/* Inside pi_task() — runs on any thread */
int tid = omp_get_thread_num();
#pragma omp atomic
tasks_per_thread[tid]++;

double pi = compute_pi(steps);
#pragma omp atomic
pi_sum += pi;

/* Spawn children if budget remains */
for (int i = 0; i < spawn_count; i++) {
    #pragma omp atomic capture
    { my_task_id = total_tasks_created; total_tasks_create
    if (my_task_id < g_num_tasks) {
        #pragma omp task firstprivate(child_seed)
        pi_task(child_seed);
    } else break;
}
```

Task Tree

T0 T1 T2 T3



AVG Π ESTIMATE
3.1416
converges with steps

SPAWN FANOUT
1 – 4
children per task

SYNCHRONISATION
atomic
pi_sum & counters

Slide 1: MPI Matrix Multiplication

Speaker: Ehtesham Faraz

The Problem

Compute **C = A × B** on a distributed memory system where N×N matrices may be too large to fit in a single node's RAM.

N × N **A** × N × N **B** = N × N **C**

1D Row-Wise Decomposition

SPMD Approach

Single Program, Multiple Data. Each rank executes the same multiplication logic on its assigned row slice.

Specifics

Each rank holds `N/P` rows of A & C. Ranks `0..N%P-1` get one extra row to handle non-divisible N.

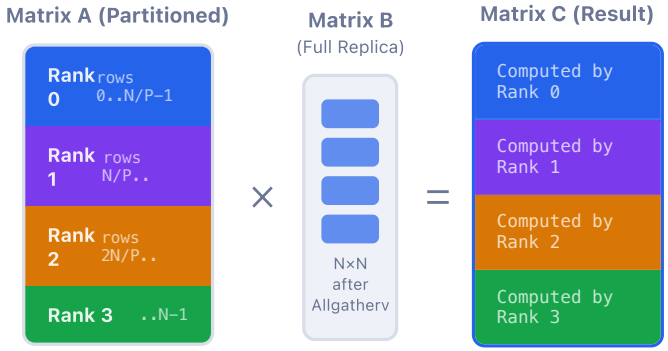
Load Balancing

Remainder rows distributed to lowest-ranked processes. All ranks differ by at most one row — near-perfect balance.

Partitioning Visualization

Ranks: P=4

Matrix A is split row-wise across ranks. Each rank generates its own slice of B, then **MPI_Allgatherv** replicates the full B. Result C is gathered to Rank 0.



mpi_matmul.c — row distribution

```
int base_rows = n / size;
int remainder = n % size;
int my_rows   = base_rows + (rank < remainder ? 1 : 0);

int my_offset = (rank < remainder)
    ? rank * (base_rows + 1)
    : remainder * (base_rows + 1) + (rank - remainder) * base_rows;
```

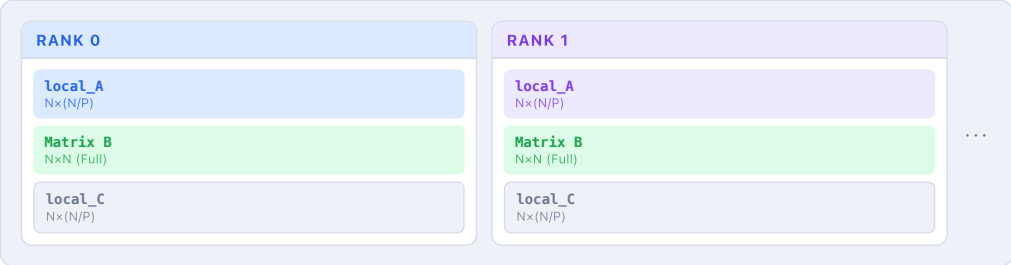
Slide 2: Memory Management & Initialization

Speaker: Ehtesham Faraz

Distributed Memory Map

local_A Full B

Allocation strategy per rank — each rank allocates only what it needs.



</> mpi_matmul.c — initialization

```c
/* Each rank generates its own A rows */
for (int i = 0; i < my_rows; i++) {
    int global_i = my_offset + i; /* global row index */
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_i, j) + seed + 0;
        local_A[i * n + j] = my_rand(&state, 0, 1);
    }
}

/* Each rank generates its own B slice */
for (int i = 0; i < my_rows; i++) {
    int global_i = my_offset + i; /* +1 offset separates A from B */
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_i, j) + seed + 1;
        local_B_slice[i * n + j] = my_rand(&state, 0, 1);
    }
}
```

Data Structures

local_A — DISTRIBUTED

Only the slice needed is allocated: `N × (N/P)`. Memory scales linearly with number of nodes.

local_B_slice — DISTRIBUTED

Each rank generates its own rows of B locally. Freed after `MPI_Allgatherv` assembles the full B.

B (full) — REPLICATED

Full N×N matrix required for multiplication. Every rank holds a complete copy after the Allgatherv phase.

Initialization Strategy

Local Generation

Data is generated locally on each node. Avoids the bottleneck of generating on Rank 0 and scattering large data over the network.

Consistent Seeding

Uses global indices `my_offset + i` to seed RNG. Matrix A uses offset `+0`, Matrix B uses `+1` — ensures independence and reproducibility.

Constraint: Verification

Only `Rank 0` allocates the full result matrix C to prevent memory overflow on worker nodes. All others discard their slice after Gatherv.

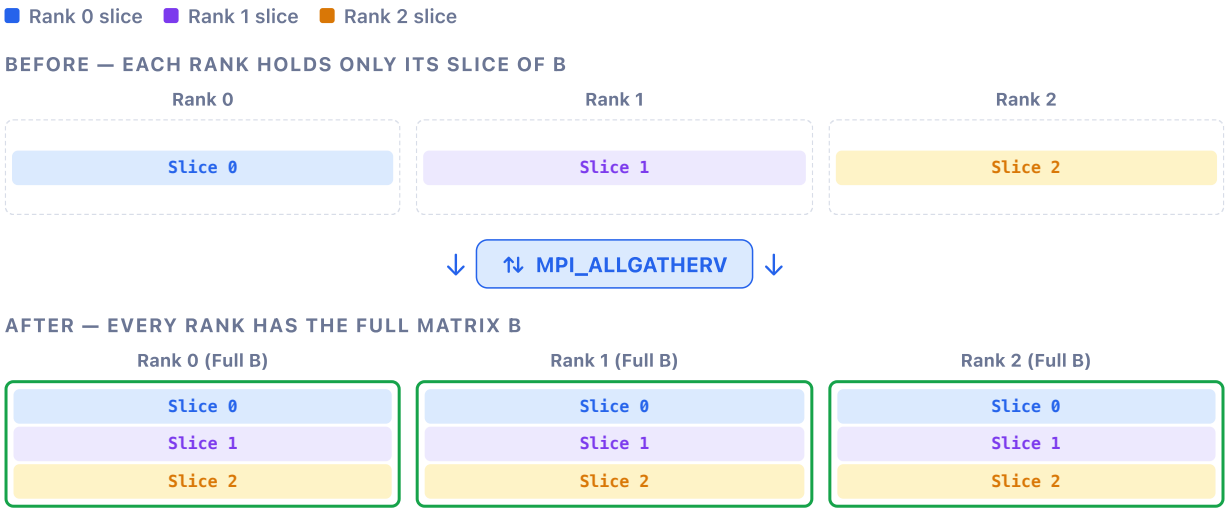
MEMORY / NODE	INIT TIME
$O(N^2/P + N^2)$	$O(N^2/P)$
slice + full B replica	per rank, fully parallel

Slide 3: Communication — MPI_Allgatherv

Speaker: Ehtesham Faraz

Collective Operation Flow

Gathering partial slices into full replicas



Implementation Insight

Decoupling communication from computation: completing data replication up-front lets the subsequent $O(N^3)$ matrix multiply proceed with zero network latency.

Strategy Details

Matrix B Replication Logic

Requirement

Every rank must possess a complete copy of Matrix B to compute its row slice of result Matrix C.

Solution

`MPI_Allgatherv` is used. Each rank initialises a unique slice of B, then the collective operation broadcasts all slices to all ranks simultaneously.

Trade-off Analysis

- ✅ **Pro:** Zero communication during the main compute phase.
- ⚠️ **Con:** Memory per node scales as $O(N^2)$. Acceptable for $N \leq 8000$ (~500 MB).

Variable-Size Displacement

Because N may not divide evenly by P, `MPI_Allgatherv` and `MPI_Gatherv` use per-rank `recvcounts` and `displs` arrays.

Rank	recvcounts[i]	displs[i]
0	$rows_0 \times n$	0
1	$rows_1 \times n$	$rows_0 \times n$
...	...	...

</> mpi_matmul.c — communication

```
/* Gather all B slices → full B on every rank */
MPI_Allgatherv(local_B_slice, my_rows * n, MPI_DOUBLE,
               B, recvcounts, displs, MPI_DOUBLE,
               MPI_COMM_WORLD);
free(local_B_slice); /* no longer needed */

/* Gather result slices → full C on Rank 0 */
MPI_Gatherv(local_C, my_rows * n, MPI_DOUBLE,
            C, recvcounts, displs, MPI_DOUBLE,
            0, MPI_COMM_WORLD);
```

Slide 4: Computation & Verification

Speaker: Ehtesham Faraz

</> mpi_matmul.c — ikj compute loop

```
/* ikj loop order: maximises cache reuse of B rows */
for (int i = 0; i < my_rows; i++) {
    for (int k = 0; k < n; k++) {
        double temp = local_A[i * n + k]; /* scalar hoist */
        for (int j = 0; j < n; j++) {
            local_C[i * n + j] += temp * B[k * n + j];
        }
    }
}

/* Collect result slices on Rank 0 for verification */
if (rank == 0) {
    double checksum = 0;
    for (long i = 0; i < (long)n * n; i++)
        checksum += C[i];
    printf("Checksum: %f\n", checksum);
}
```

📈 **OBSERVATION**

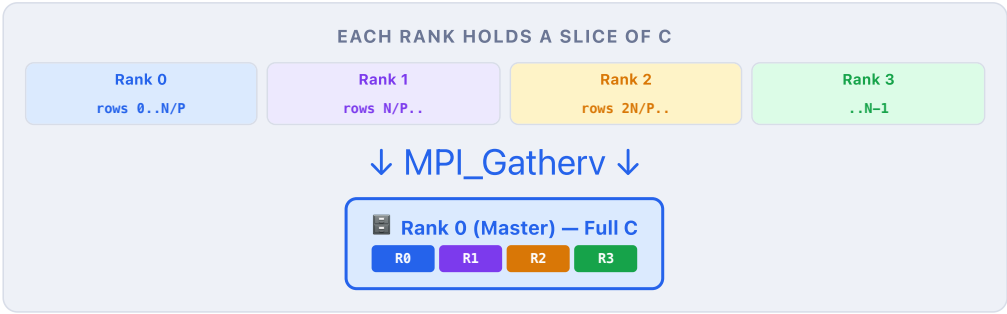
ikj order keeps B rows in L1/L2 cache across the inner j-loop, greatly reducing cache misses vs. naive ijk.

⚠️ **LIMITATION**

Full B replica (N×N per node) limits scalability. For very large N, a Cannon's algorithm avoids this overhead.

📩 MPI_Gatherv — Collect to Rank 0

Each rank sends its computed slice of C. Only Rank 0 allocates the full result matrix.



✅ Verification Methodology

- **Checksum Calculation**
  Rank 0 sums all N^2 elements of C using `double` arithmetic. Compared against a sequential baseline for correctness.

- **Validation Status**
  Checksum matches sequential reference across all tested configurations.
  ✓ PASSED — Checksum matches sequential baseline

COMPUTE / NODE	COMMUNICATION
$O(N^3/P)$	$O(N^2)$
ideal linear speedup	one-time Allgatherv

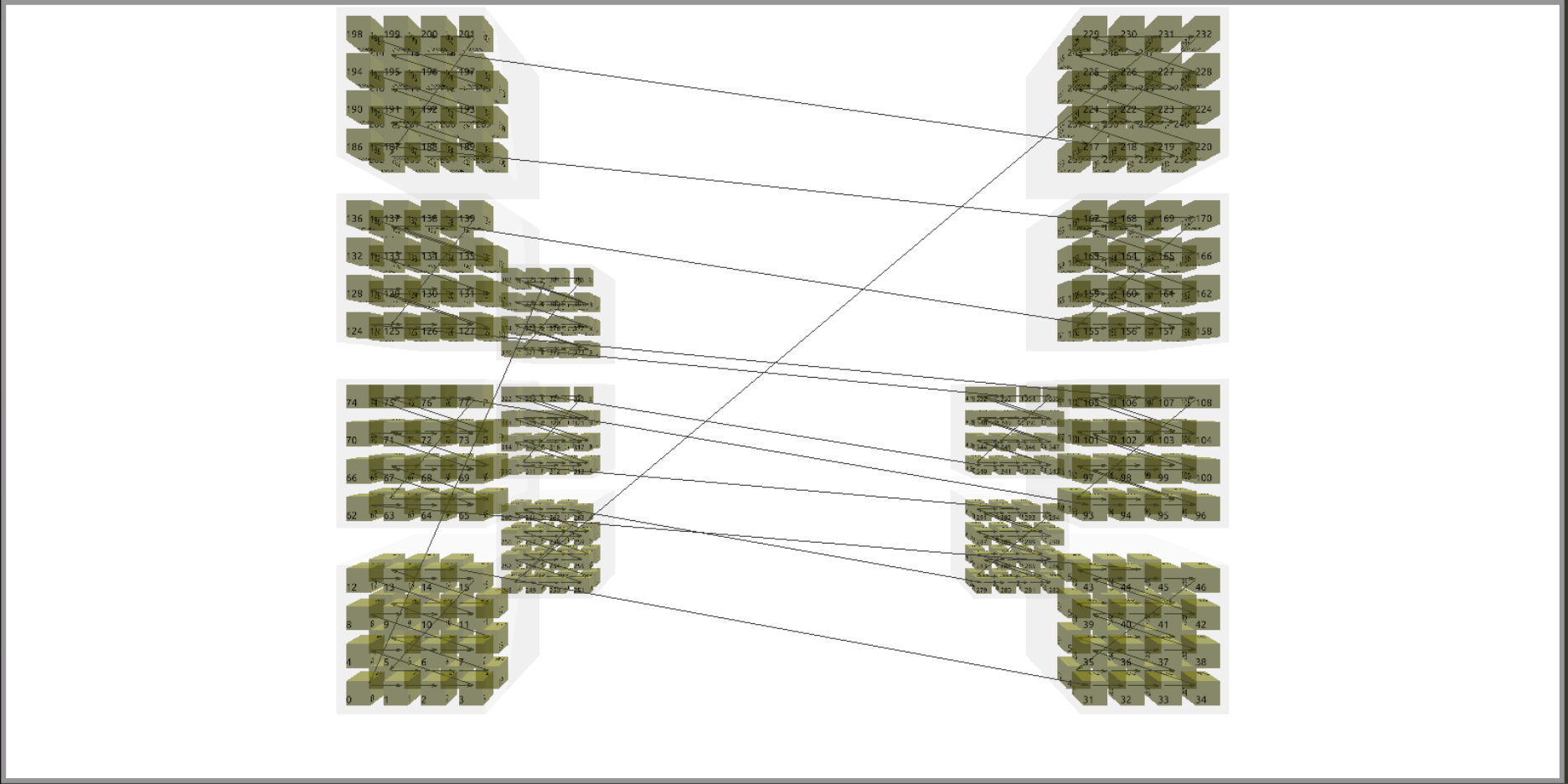
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File Folder

Upload)ownloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

3D Nodes (Cube View)

2D Nodes (List View)

Communication Matrix

Show detailed MPI communication patterns:

P2P - Send arrows

P2P - Recv arrows

Collective arrows

Runtime Options

Screenshot

time: 0 s MPI functions: MPI_Allgatherv (ring),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

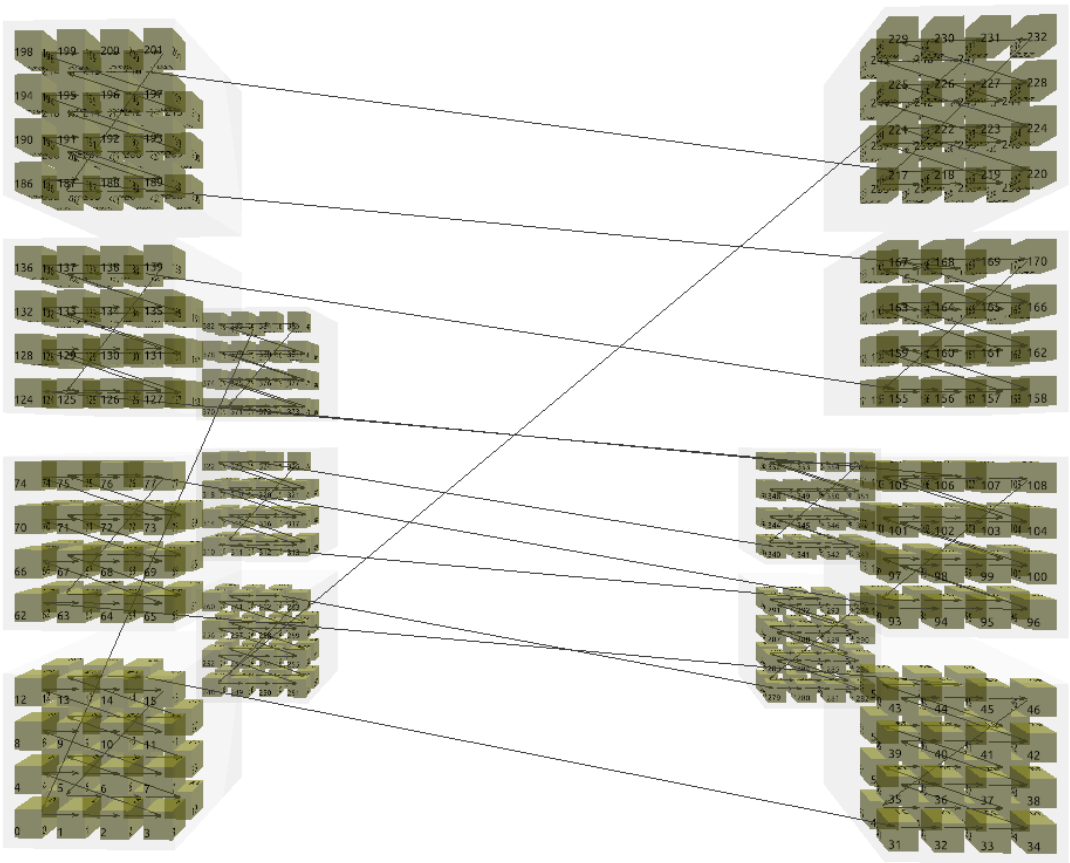
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File | Folder

Upload /home/ubuntu/Downloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 1 s MPI functions: MPI_Allgatherv (ring),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

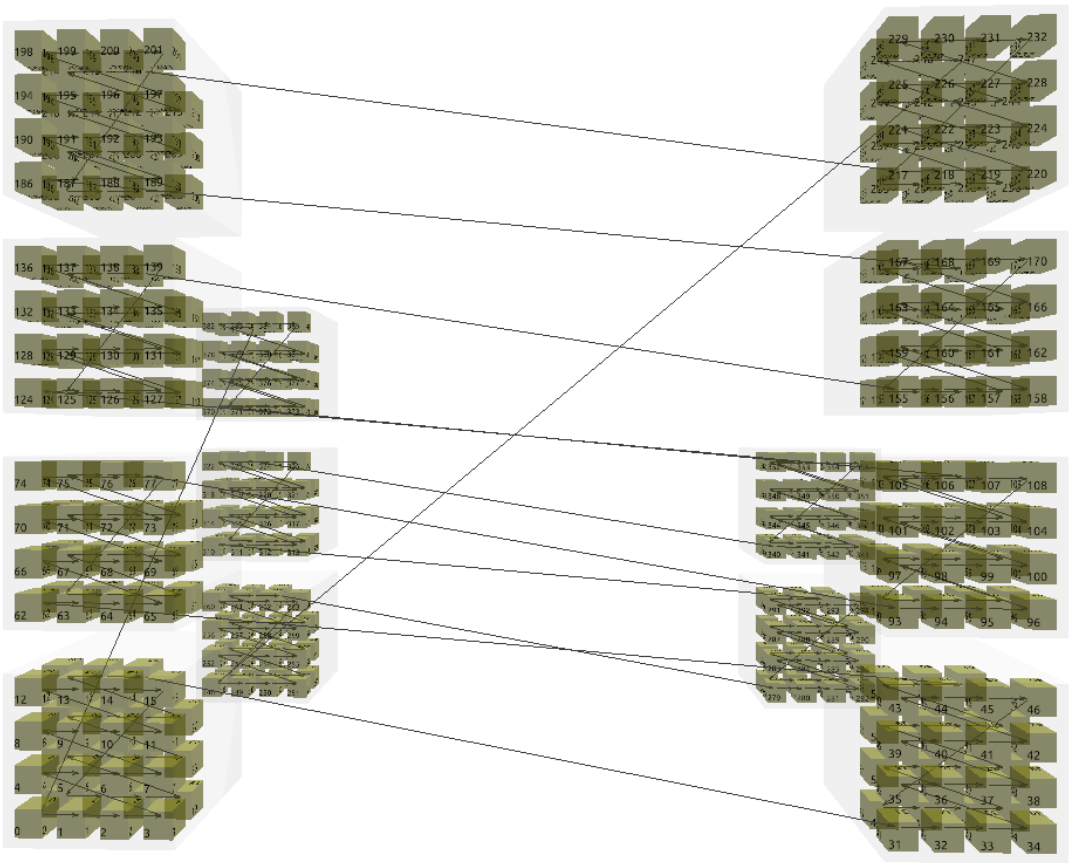
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File | Folder

Upload | Downloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 2 s MPI functions: MPI_Allgatherv (ring),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

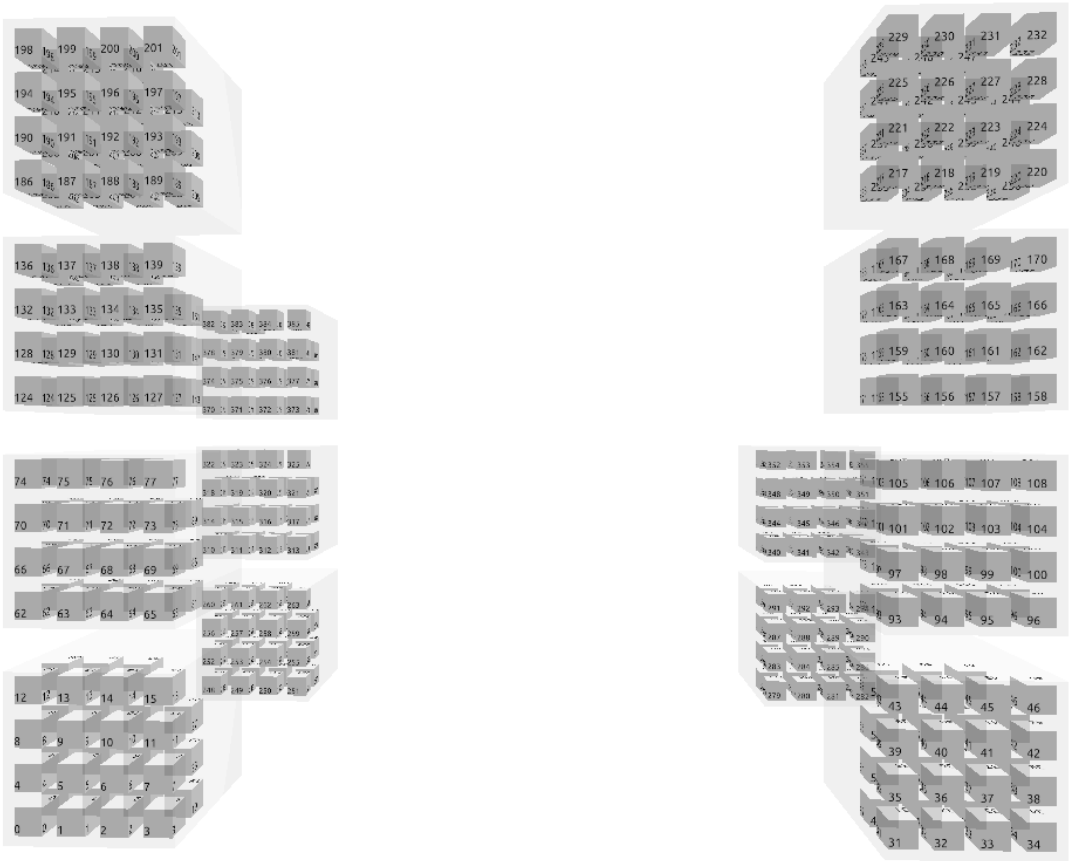
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File | Folder

Upload /ownloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 3 s MPI functions:

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

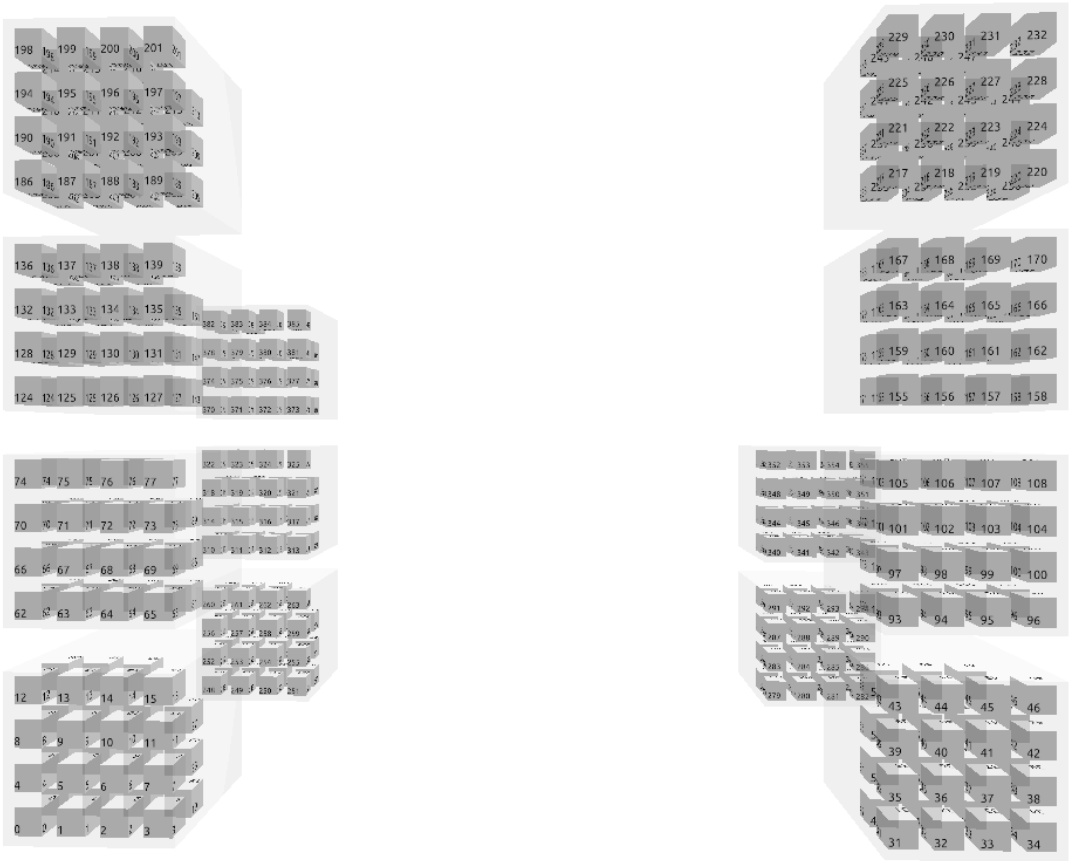
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File | Folder

Upload /home/ubuntu/Downloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 4 s MPI functions:

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

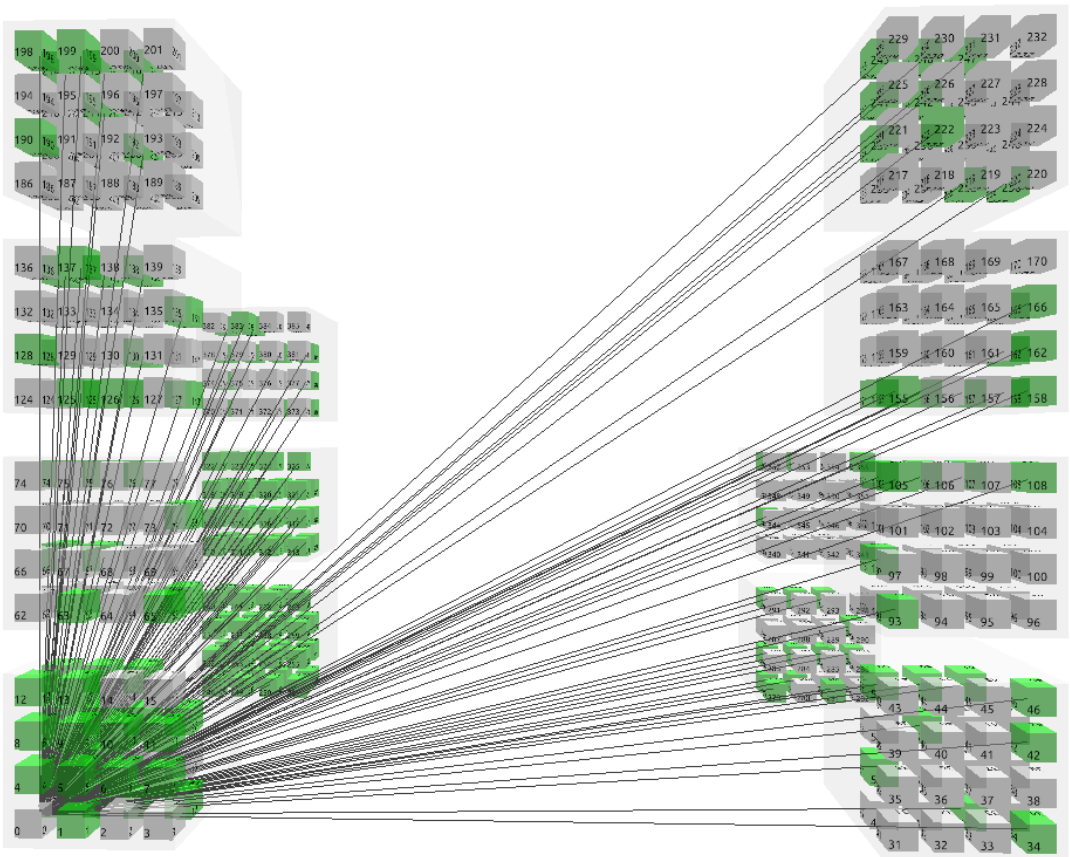
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File Folder

Upload)ownloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 5 s MPI functions: MPI_Gatherv (),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

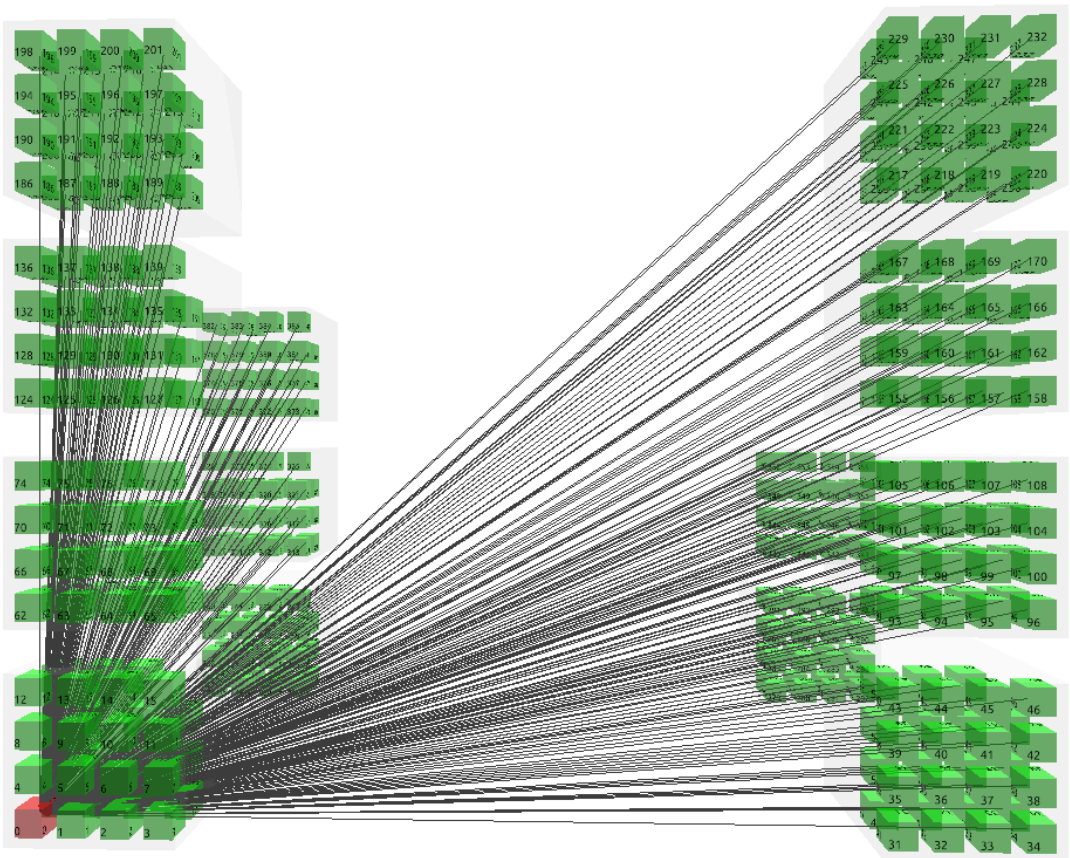
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File Folder

Upload)ownloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

3D Nodes (Cube View)

2D Nodes (List View)

Communication Matrix

Show detailed MPI communication patterns:

P2P - Send arrows

P2P - Recv arrows

Collective arrows

Runtime Options

Screenshot

time: 6 s MPI functions: MPI_Gatherv (),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

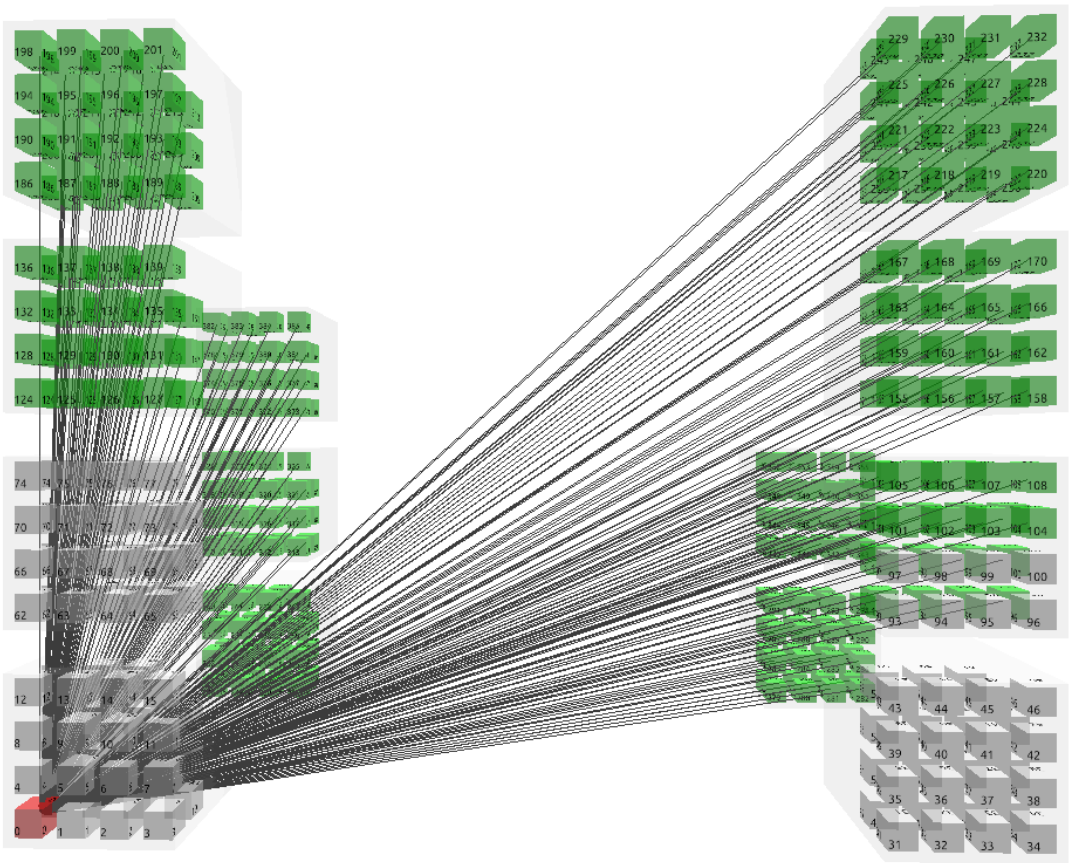
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File Folder

Upload)ownloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 7 s MPI functions: MPI_Gatherv (),

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

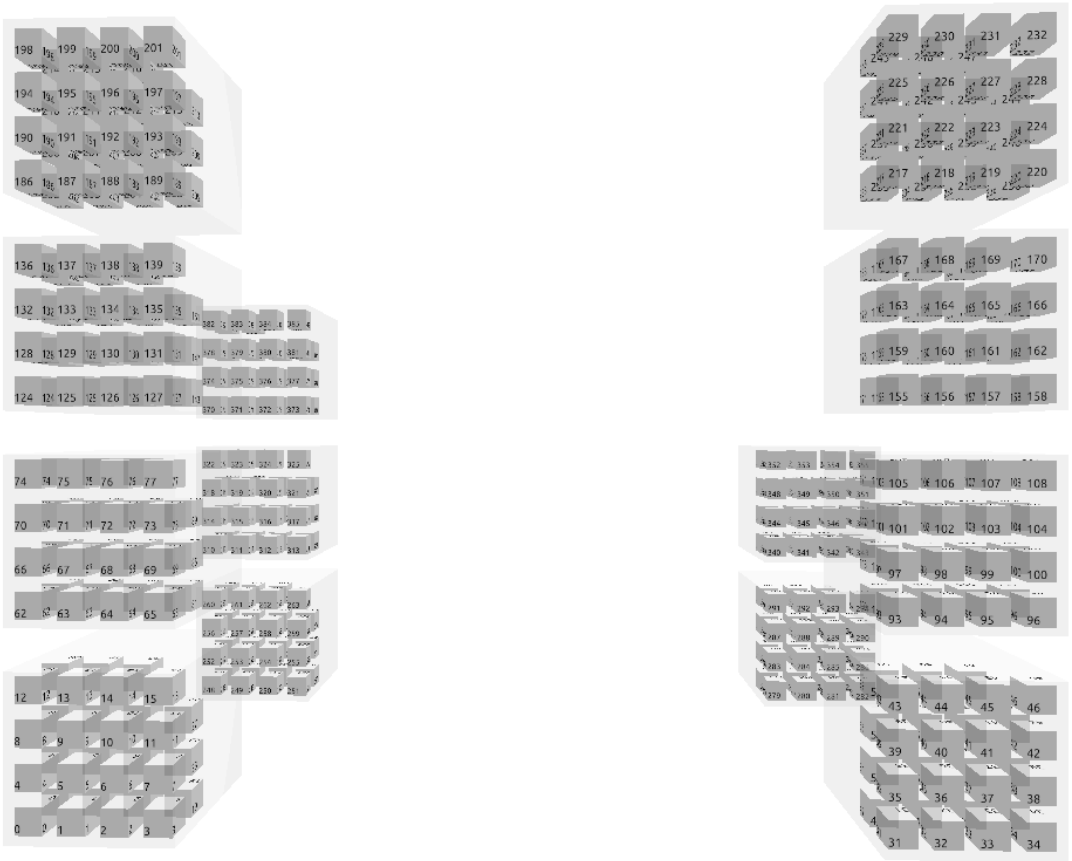
EduMPI

Settings Help Non-Visualization

EduMPI-Run ID: 3891

3D Cluster Overview: Send/Recv Ratio per Process

serial comput./idle | mainly send | same send/recv | mainly recv



MPI Application

Select MPI Application

Select a .c file or a folder containing your program

.c File | Folder

Upload /home/ubuntu/Downloads/matmul.c

name of the MPI program:

matmul .c

Number of Processes: 400

Start MPI-Program

Cancel MPI-Program

Views

Select representation view:

- [x] 3D Nodes (Cube View)
- [ ] 2D Nodes (List View)
- [ ] Communication Matrix

Show detailed MPI communication patterns:

- P2P - Send arrows
- P2P - Recv arrows
- Collective arrows

Runtime Options

Screenshot

time: 8 s MPI functions:

13:49:18 13:49:23 13:49:28 13:49:33 13:49:38 13:49:43 13:49:48 13:49:53 13:49:58 13:50:03 13:50:08 13:50:13 13:50:18 13:50:23 13:50:28 13:50:33 13:50:38 13:50:43 13:50:48 13:50:53 13:50:58 13:51:03 13:51:08

Slide 1: Matrix Partitioning & Load Balancing

Speaker: Fuzail

The Problem

Compute **C = A × B** on a distributed memory system where matrices may be too large to fit in a single node's RAM.

N × N **A** × N × N **B** = N × N **C**

Approach : 1D Row Wise Decomposition

SPMD Approach

Single Program, Multiple Data. Each rank executes the same multiplication logic on its assigned row slice.

Load Balancing

If `n % size != 0`, rows cannot be evenly divided. Some ranks will naturally be assigned more work than others.

My Solution

Base rows: `n / size`

The first `remainder` processes receive exactly 1 extra row.

```
rows_per_proc = n / size + (i < remainder ? 1 : 0);
```

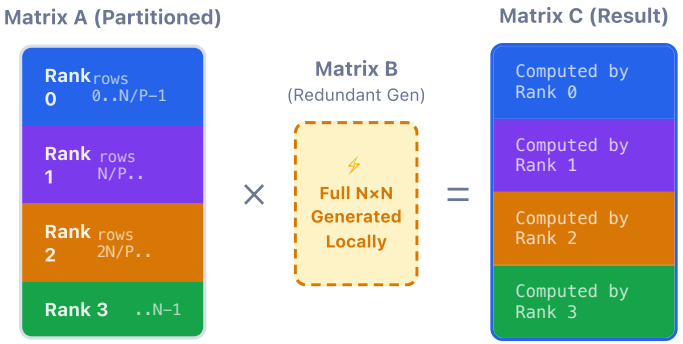
Why This Is Balanced

- **Negligible Imbalance:** Maximum difference between any two processes is at most 1 row. For large `n` (e.g., 8000), this is mathematically insignificant.
- **No Idle Time:** Ensures an incredibly even workload distribution.
- **Good Scalability:** Prevents faster nodes from idling at the final implicit barrier.

Partitioning Visualization (Zero-Comm)

Ranks: P=4

Matrix A is split row-wise across ranks. Matrix B is **fully generated locally on every rank** using redundant CPU cycles.



matmul.c — row distribution & displacement calculation

```c
for (int i = 0; i < size; i++) {
    int rows_per_proc = n / size + (i < remainder ? 1 : 0);
    sendcounts[i] = rows_per_proc * n;
    displs[i] = sum;

    if (i == rank) {
        my_rows = rows_per_proc;
        my_row_start = sum / n;  // Used to offset PRNG seed
    }
    sum += sendcounts[i];
}
```

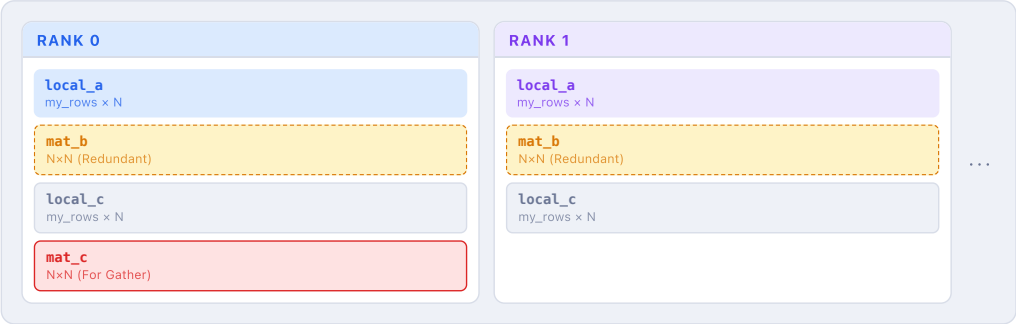
Slide 2: Memory Management& Initialization

Speaker: Md Fuzail

Streamlined Memory Map

local_a mat_b

No intermediate slices. Memory is allocated strictly for computation.



matmul.c — initialization

```c
/* EVERY rank generates full matrix B locally */
double *mat_b = alloc_matrix(n, n);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(i, j) + seed + 1;
        mat_b[i * n + j] = my_rand(&state, 0, 1);
    }
}

/* Each rank generates its specific A rows using offset */
double *local_a = alloc_matrix(my_rows, n);
for (int i = 0; i < my_rows; i++) {
    int global_row_idx = my_row_start + i;
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_row_idx, j) + seed + 0;
        local_a[i * n + j] = my_rand(&state, 0, 1);
    }
}
```

Data Structures

local_a — DISTRIBUTED

Only the slice needed is allocated: `my_rows × N`. Scales perfectly with node count.

mat_b (Full) — REDUNDANT GEN

No intermediate `slice` buffers. The full N×N matrix is generated straight into RAM using local CPU cycles.

local_c — RESULT BUFFER

Pre-initialized to zero using `memset`. Stores the local partial product of the multiplication.

1D Contiguous Allocation

All matrices use flat 1D arrays (`malloc(rows * cols)`). This prevents memory fragmentation, eliminates pointer-chasing, and guarantees cache locality.

Zero-Comm Strategy

Network Bypass

By abandoning collective communication completely, the cluster network bandwidth consumption is strictly **zero** during startup.

Offset Math

Nodes use `my_row_start + i` to independently seed the RNG. This guarantees all nodes compute the exact correct slice of Matrix A.

Constraint: Master Memory

Rank 0 must allocate a massive final `mat_c` (N×N) array just before `MPI_Gatherv` to receive and checksum the data.

MEMORY / NODE	INIT TIME
$O(N^2/P + N^2)$	$O(N^2)$
slice + full B replica	Fixed redundant CPU cost

Slide 3: Cache Optimization, Final Gather & Verification

Speaker C: Fuzail

⚡ i-k-j Compute Core

With data generated, isolated math begins immediately.

matmul.c — Cache Tiling

```
// Cache-friendly i-k-j loop order
for (int i = 0; i < my_rows; i++) {
    for (int k = 0; k < n; k++) {
        double a_val = local_a[i * n + k];
        for (int j = 0; j < n; j++) {
            local_c[i * n + j] +=
                a_val * mat_b[k * n + j];
        }
    }
}
```

🧠 **SPATIAL LOCALITY GUARANTEE**

By hoisting `a_val` and iterating over `j` internally, the hardware prefetcher reads `mat_b` linearly (stride-1). This drastically reduces L1/L2 cache misses compared to naive i-j-k loops.

📩 The Only Comm.

Network utilization remains strictly at **0%** until this exact line of code.

matmul.c — Final Gather

```
// Rank 0 allocates master buffer
double *mat_c = NULL;
if (rank == 0) mat_c = alloc_matrix(n,n);

MPI_Gatherv(local_c, my_rows * n, MPI_DOUBL
            mat_c, sendcounts, displs,
            MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

SENDING LOCAL_C SLICES

Rank 0	Rank 1	Rank 2	Rank 3
0..N/P	N/P..	2N/P..	..N-1

↓ Gatherv ↓

🗄 Rank 0 (Master) — mat_c

R0 | R1 | R2 | R3

✅ 100% Deterministic

Once Gathered, Rank 0 verifies mathematical integrity against the baseline.

matmul.c — fmod Verify

```
if (rank == 0) {
    double checksum = 0.0;
    for (int i = 0; i < n * n; i++) {
        checksum += mat_c[i];
    }

    // Constrain checksum bounds
    checksum = fmod(checksum,
                    18446744073709551616.0);

    printf("Checksum: %f\n", checksum);
}
```

- **fmod Constraining**
  By applying a 64-bit floating-point modulo (`fmod`), we ensure massive results do not overflow, allowing exact 1:1 validation.

✓ Matches Assigned Constraints perfectly

Slide 4: Strategy Summary & Visualizations

Speaker : Fuzail

Hierarchical Architecture Overview

1. 1D Row Decomposition

A strict SPMD approach where the Matrix A workload is statically sliced horizontally across completely independent parallel worker nodes.

2. The Core Hypothesis

HPC networks are volatile. I hypothesized that burning local CPU cycles to generate data would bypass cluster network bottlenecks completely.

3. Memory Management

Avoided pointer arrays. All matrices strictly use 1D Contiguous Allocation (`malloc(rows * cols)`) to prevent memory fragmentation.

4. Perfect Load Balancing

Used deterministic offsets to cleanly distribute remainder rows to lower ranks. Maximum imbalance between any two nodes is strictly 1 row.

5. Cache Optimization

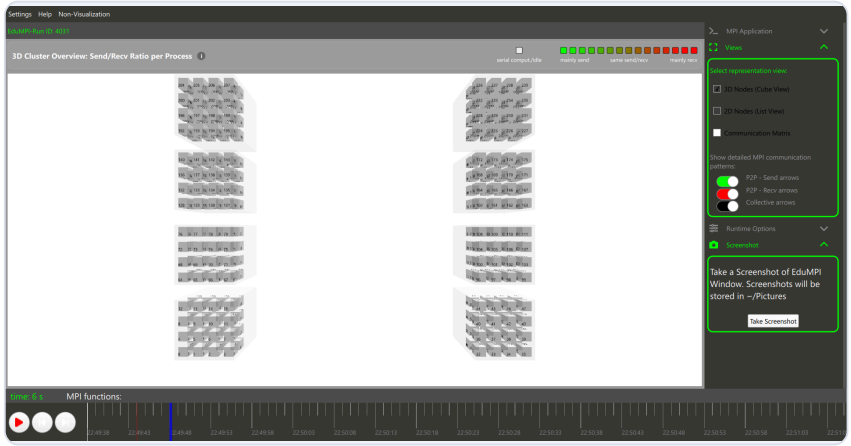
Refactored the compute loop to an `i-k-j` order. This enables linear stride-1 fetching of Matrix B, maximizing L1/L2 cache hits.

6. Zero-Comm Trade-off

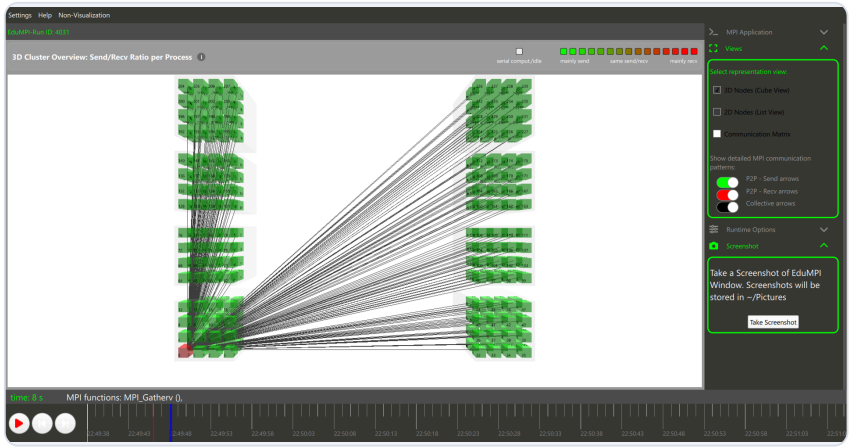
Achieved 0% network usage during setup by trading bandwidth for redundant `O(N²)` local PRNG generation of Matrix B on every node.

EduMPI Visualizations

Grey State: Isolated computation with 0% network utilization



Clean Gather: A single MPI_Gatherv operation at program exit



Slide 5: Architectural Comparison

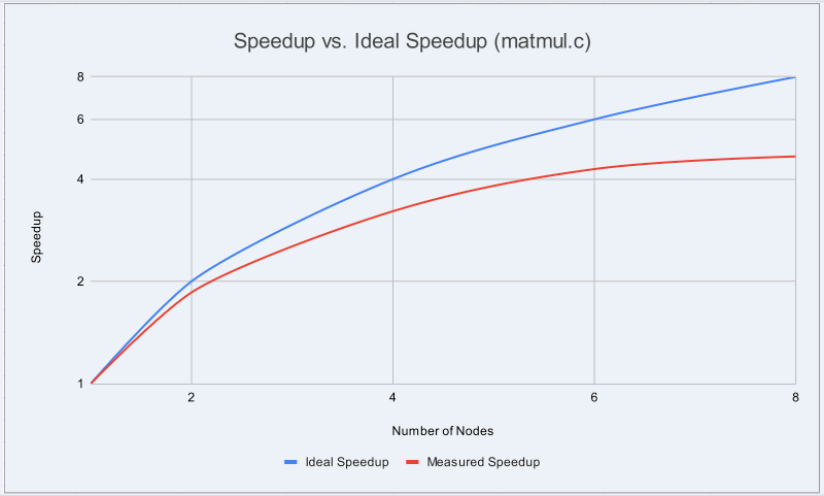
Speaker : Fuzail

Approach A MPI_ALLGATHERV

The Hypothesis: The CPU cost of generating 64 million random numbers via complex bitwise shifts is too heavy.

The Technical Implementation: Divide the PRNG generation workload proportionally across all ranks ($O(N^2/P)$). Once each node computes its slice, rely entirely on the high-speed cluster network (`MPI_Allgatherv`) to stitch the full 512 MB Matrix B together across all nodes.

Approach A Speedup Curve



Approach B ZERO-COMM

The Hypothesis: Transmitting 512 MB across a shared, multi-tenant university cluster will cause massive network congestion and unpredictable latency.

The Technical Implementation: Completely bypass the network. Accept a redundant computational penalty where every single CPU independently generates the entirety of Matrix B locally ($O(N^2)$). Achieve 0% network bandwidth consumption until the final verification gather.

Approach B Speedup Curve

